



PROGRAMACIÓN I

Pseudocódigo

María Janneth Rivera Reyna

Licenciatura en Ciencias de la Computación

Departamento de Matemáticas

Universidad de Sonora

Ciclo 2026-2

Pseudocodigo

Es una **técnica** que nos permite **describir los pasos** para la **resolución de un problema**.

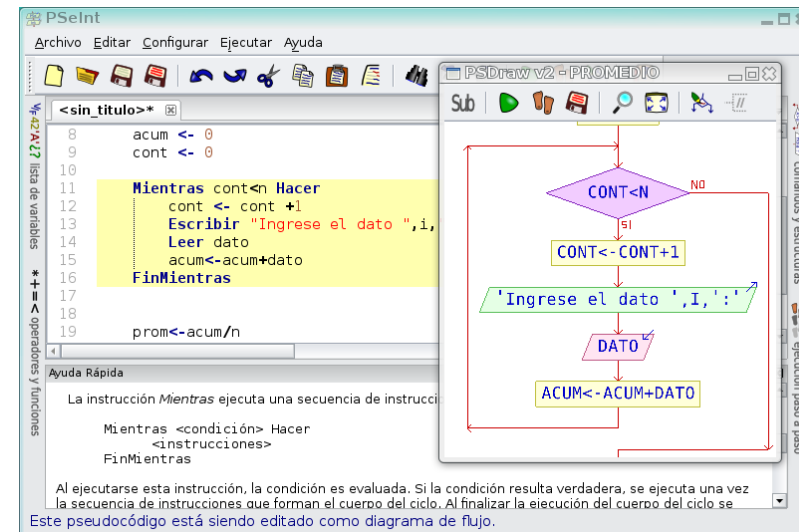
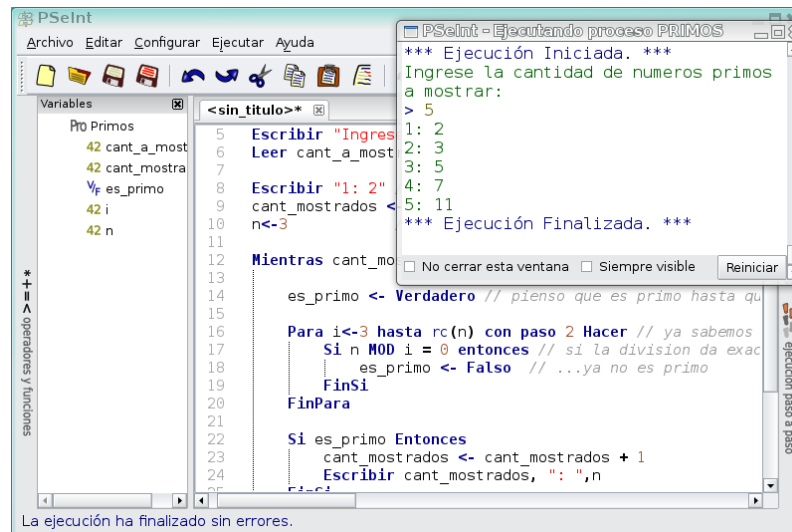
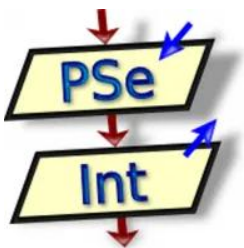
No se considera un lenguaje de programación, sino más bien **es un lenguaje intermedio** entre el lenguaje natural humano y el lenguaje de computadora:

- Es lo suficientemente **parecido a al lenguaje natural** (en nuestro caso, al español)
- Requiere del **uso de ciertas reglas** para expresar las ideas de una forma lógica, pero sin preocuparnos por la sintaxis de un lenguaje de programación.

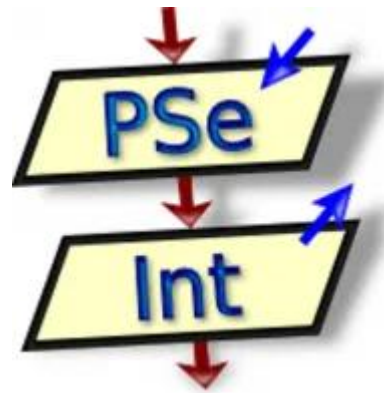
No se puede ejecutar en una computadora, sin embargo **existen programas didácticos** que nos ayudan a **escribir, probar y ejecutar algoritmos** y ver que funcionen de forma correcta.

PSeInt

- Software libre educativo multiplataforma dirigido a personas que inician en programación.
- Creado por Pablo Novara en 2003, como proyecto final de la materia de Programación I, en la Universidad Nacional del Litoral, en Argentina.
- Es utilizado principalmente por estudiantes para aprender los **fundamentos de la programación y el desarrollo de la lógica, usando pseudocódigo y diagramas de flujo.**



Elementos básicos en PSeInt



VARIABLE

Es un **espacio en la memoria RAM** de la computadora que sirve para **almacenar un dato o valor**.

Tiene 3 atributos:

- **Nombre o Identificador:** Permite identificar un dato para distinguirlo de otros y poder manipularlo.
- **Contenido:** Es el valor que contiene el dato.
- **Tipo:** El tipo de dato asociado.

IDENTIFICADORES

Nombres usados para hacer **referencia a las variables**.

REGLAS

- Puede contener sólo:
 - Mayúsculas y minúsculas [a-z] [A-Z], excepto ñÑ
 - Dígitos [0-9]
 - Caracter de subrayado (_)
- Puede iniciar con letra o (_), no con dígito
- No debe ser una palabra reservada de PSeInt
- No hace distinción entre mayúsculas y minúsculas
- No debe contener espacios

Palabras reservadas en PSeInt

Algoritmo	FinAlgoritmo	Proceso	FinProceso
Definir	Como	Entero	Enteros
Caracter	Caracteres	Cadena	Cadenas
Leer	Si	Entonces	SiNo
FinSi	Segun	Hacer	De Otro Modo
FinSegun	Mientras	FinMientras	Repetir
Hasta Que	Para	Con Paso	FinPara
Funcion	FinFuncion	Fin	Verdadero
Falso	Y	O	No
Mod	Div	Limpiar Pantalla	Esperar Tecla
Dimension	Mostrar	Imprimir	es
Son	Real	Reales	Texto
Logico	Logicos	Numero	Numerico
Logica	Logicas	Entera	Enteras
Escribir	Sin	Saltar	Limpiar
Pantalla	opcion	valor	

IDENTIFICADORES

Identificador	Válido	Detalle
suma enteros		El espacio en blanco
Año		La ñ
calificación		La letra ó con acento
2c		Iniciar con dígito
precio-total		El guión
nombreUsuari@		La @
Nombre.apellido		El punto
a+b		El +
_contador		Es válido
RESULTADO_1		Es válido

TIPOS DE DATOS SIMPLES

PSelnt soporta 3 tipos de datos simples:

- **Numérico:** Almacena valores numéricos enteros o reales (usan el punto para separar decimales). Para definir este tipo de dato se usa **Número**, **Numérico**, **Real**, **Entero**.
- **Lógico:** Puede contener sólo los valores **FALSO** y **VERDADERO**, aunque también puede usarse **F** y **V**, ó **0** y **1**.
- **Cadena:** Pueden contener un caracter o una cadena de caracteres encerrados entre comillas (simples o dobles). Se define como **Caracter**, **texto**, **cadena**.

TIPOS DE DATOS SIMPLES

Tipo de dato	Ejemplos
Entero	<i>1, 4, -6, 342, -32021, 0</i>
Real	<i>-238.423, 2.0, 3.1542, 0.00001</i>
Caracter	<i>"{", "*", "9", "Universidad de Sonora", "Mi teléfono es +52 (662) 123-45-67", 'Hola', 'S'</i>

DECLARACIÓN DE VARIABLES

- Para poder utilizar una variable en un algoritmo, es necesario **declararla**.
- Cuando declaramos una variable, le estamos anunciando a la computadora **qué tipo de datos** vamos a almacenar en dicha variable.

FORMATO

DEFINIR <var> **COMO** [tipo singular]

DEFINIR <var1, var2, ...> **COMO** [tipo plural]

Ejemplos:

Definir edad **Como Entero**

Definir edad1, edad2 **Como Enteros**

BUENAS PRÁCTICAS

IDENTIFICADORES

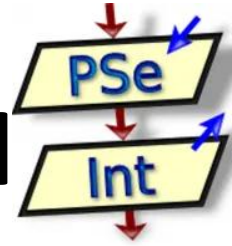
- Usar identificadores que estén relacionados con lo que representan en el algoritmo para que sea mas legible.

Ejemplo:

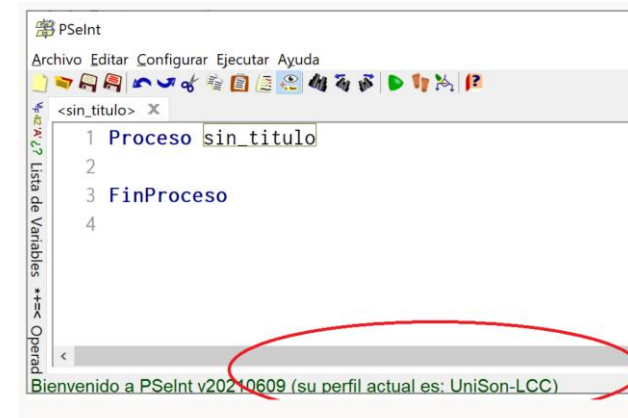
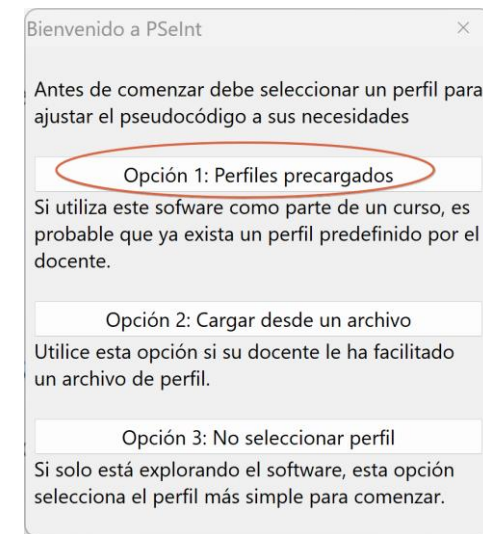


- Si se va a declarar una variable que guarde una calificación, podría usarse el identificador “calif” o bien, “calificacion”.
Usar “c”, “c1”, “clf” podría no ser muy intuitivo.

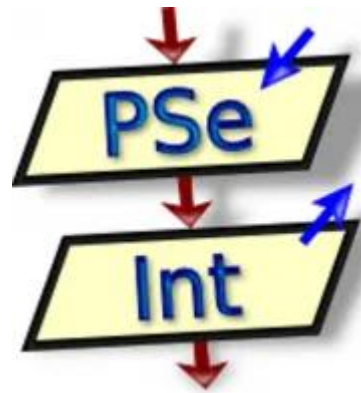
PSeInt: Instalación y configuración del perfil



1. Visita la página de [PSeInt](#) y descarga para tu sistema operativo.
2. Si es la primera vez que lo instalas en tu computadora, verás el siguiente menú, elige **Opción 1: Perfiles precargados**.
3. Si ya lo tienes instalado, ve al menú **Configurar > Opciones del lenguaje (perfiles)**
4. En **Buscar**, escribe: **UniSon-LCC** y selecciona. Haz click en **Aceptar**.
5. Asegúrate que en la parte inferior el perfil actual sea Unison-LCC.
6. **Como evidencia**, sube en tu tarea asignada en Teams, una captura de tu pantalla como la siguiente:



Ejercicios: Declaración de variables



¡Manos a la obra!

ESCRITURA DE DATOS

Esta operación permite a la máquina presentarle datos al usuario.

FORMATO

ESCRIBIR <var>

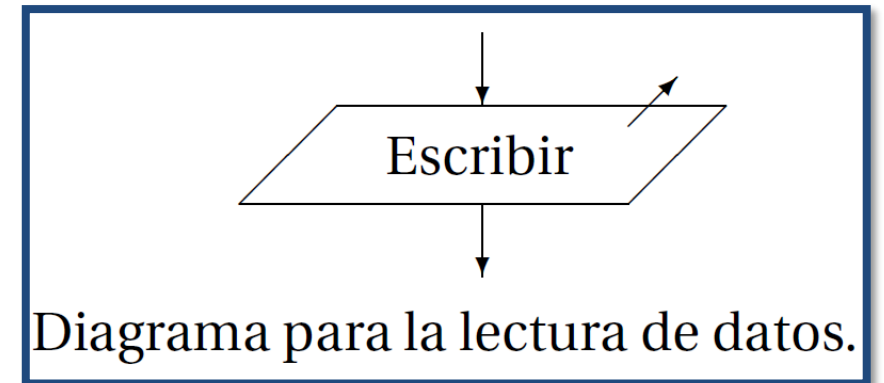
Ejemplo:

ESCRIBIR dato1

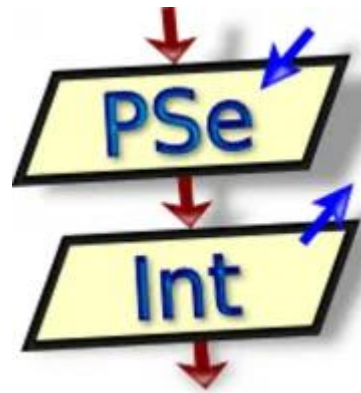
Donde:

ESCRIBIR → especifica la operación a realizar

'dato1' → es el nombre de la variable que contiene
la información que se quiere presentar al usuario



Ejercicios: Escritura de variables



¡Manos a la obra!

LECTURA DE VARIABLES

Con esta operación se captura o se recibe la información dada por el usuario.

FORMATO

LEER <var>

Ejemplo:

LEER dato1

Donde:

LEER → especifica la operación a realizar

'dato1' → es el nombre de la variable donde se almacenará la información leída.

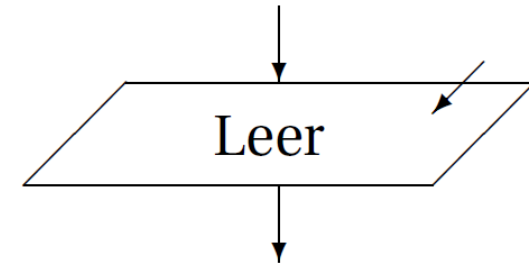
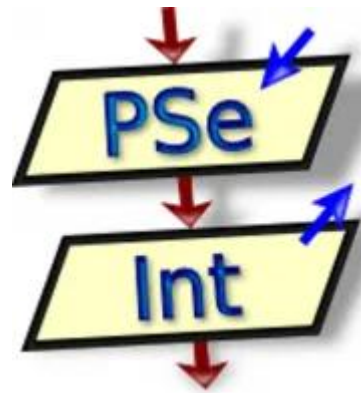


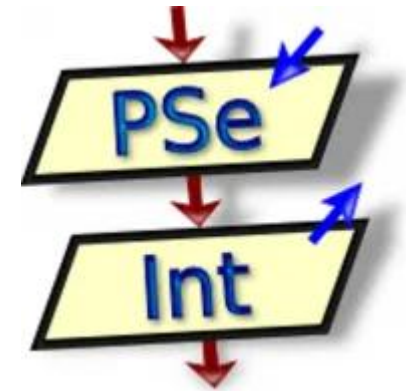
Diagrama para la lectura de datos.

Ejercicios: Lectura de variables



¡Manos a la obra!

Práctica 1



- En PSeInt, escribe el pseudocódigo para preguntarle al usuario la siguiente información:
 - Nombre
 - Edad
 - Carrera que cursa
 - Hobbies / Pasatiempo favorito
 - Usuario de correo (por ejemplo: “janneth”)
 - Dominio del servidor de correo (por ejemplo: “@unison.mx”)
 - Mostrar texto: “Tu correo es: janneth@unison.mx”

OPERADORES ARITMÉTICOS

Operación	Operador
Suma	+
Resta	—
Multiplicación	*
División (real)	/
Módulo	MOD
Asignación	:= <-

- Todos los operadores (excepto el de asignación) se aplican de modo *binario* (con dos operadores) y utilizan operandos constantes o variables a la izquierda y a la derecha del operador:

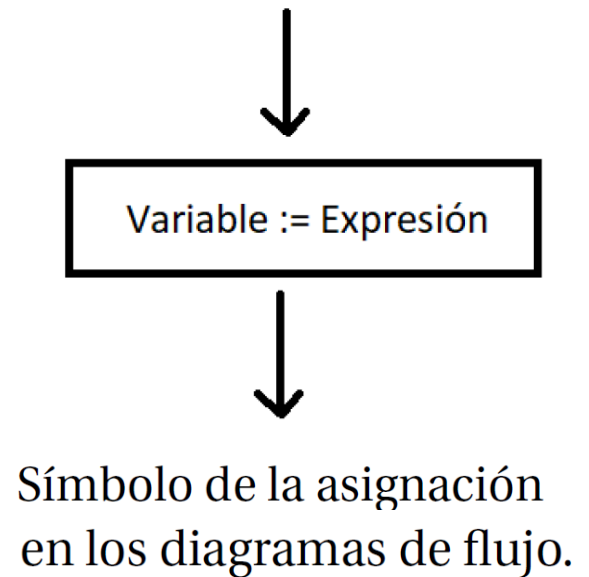
- 5+4
- b-a
- a*2
- 20/5
- 20MOD4

OPERADORES ARITMÉTICOS

- El operador de **asignación** sirve para asignar el valor de la derecha a la variable de la izquierda:

variable := valor asignado

x := 2	
x := y	
x := 2+y	
x := aleatorio(1,5)	
letra := 'a'	
cad := 'Juan'	



OPERADORES ARITMÉTICOS

Los operadores $+$ y $-$ también pueden ser aplicados en forma *monaria*, es decir, con un operando.

Por ejemplo:

$x1 := -2$	asigna el valor constante -2 a x1
$x2 := +3$	asigna el valor constante 3 a x2

OPERADORES ARITMÉTICOS

Las operaciones deben escribirse en una sola línea.

Por ejemplo, si se tiene la fórmula:

$$x = \frac{1}{a} + \frac{1}{b}$$

debe transformarse para que tome la forma:

$$x = 1/a + 1/b$$

OPERADORES ARITMÉTICOS

Hay ocasiones en que la transformación no es tan directa.

Por ejemplo, la ecuación:

$$x = \frac{a + b}{2} c$$

puede transformarse a:

$$\begin{aligned} x &= (a + b) * c / 2 \\ x &= (a + b) / 2 * c \end{aligned}$$

OPERADORES ARITMÉTICOS

Para hacer la transformación correcta, es necesario conocer la **precedencia de los operadores**, es decir, saber qué operador se debe ejecutar primero:

Nivel de precedencia	Operadores	Asociatividad
1	()	
2	+ - (monarios)	Derecha a izquierda
3	* / MOD	Izquierda a derecha
4	+ - (binarios)	Izquierda a derecha
5	:=	Derecha a izquierda

Los que están hasta arriba de la tabla tienen mayor precedencia. Si se quiere modificar la precedencia, se utilizan los paréntesis.

Por ejemplo:

$$x = \frac{2 + 5a}{-\frac{a}{2} + ab}$$

Se transformaría a:

$$x = (2 + 5 * a) / (-a / 2 + a * b)$$

Ejercicio: Calcular el área de un triángulo

$$A = \frac{ba}{2} \longrightarrow a := base * altura / 2$$

1. **Identifiquen:**
 - a. Entrada
 - b. Proceso
 - c. Salida
2. **Dibujen** el diagrama de flujo.
3. **Escriban** el pseudocódigo de este algoritmo.



COMENTARIOS

Los comentarios son **notas explicativas** pensadas para ser leídas por un futuro lector (que puede ser alguien más o el mismo programador).

El objetivo es hacer **más legible el código**, sin afectar su funcionamiento, ya que este texto no se ejecuta.

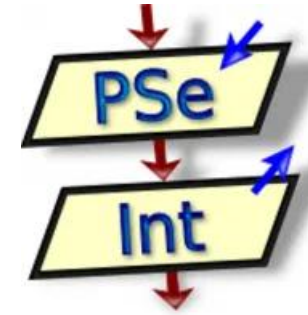
FORMATO

```
//Este es mi primer comentario
```

Donde:

Todo lo que quede a la derecha del símbolo `//` se considera comentario hasta el final de esa línea.

Práctica 2



En PSeInt:

1. **Escribe** el pseudocódigo para calcular el área de un triángulo. Utiliza como base **el diagrama de flujo y el pseudocódigo** realizado anteriormente en clase.
2. **Escribe comentarios en la cabecera del programa:**
 - a) Nombre del archivo
 - b) Autor
 - c) Descripción
3. **Comenta cada línea** para indicar lo que se está haciendo.
4. **Crea y guarda el diagrama de flujo** generado por PSeInt. ¿Se parece al que hiciste en clase?

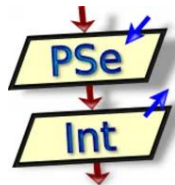
Práctica 3a: Calcular el área de un círculo

$$A = \pi * r^2 \longrightarrow A = pi * radio * radio$$



En su cuaderno:

1. **Identificar:**
 - a. Entrada
 - b. Proceso
 - c. Salida
2. **Dibujar** el diagrama de flujo.
3. **Escribir** el pseudocódigo de este algoritmo.

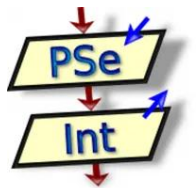


En PSeInt:

1. **Escribir** el pseudocódigo de este algoritmo.
Leer un valor aproximado para pi (p.ej. 3.14)
2. **Agregar comentarios en cabecera.**
 - a) Nombre del archivo
 - b) Autor
 - c) Descripción
3. **Comentar cada línea** describiendo lo que se está haciendo.
4. **Generar y guardar diagrama de flujo.**

Práctica 3b: Calcular el área de un círculo

$$A = \pi * r^2 \longrightarrow A = PI * radio \uparrow 2$$



En PSeInt:

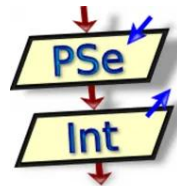
1. Repetir pseudocódigo anterior, pero ahora utilizando:
 - a) La constante **PI** definida en PSeInt
 - b) El operador de **potencia** $\uparrow$

Práctica 4: Calcular el promedio de 3 calificaciones



En su cuaderno:

1. **Identificar:**
 - a. Entrada
 - b. Proceso
 - c. Salida
2. **Dibujar** el diagrama de flujo.
3. **Escribir** el pseudocódigo de este algoritmo.



En PSeInt:

1. **Escribir** el pseudocódigo de este algoritmo.
2. **Agregar comentarios en cabecera.**
 - a) Nombre del archivo
 - b) Autor
 - c) Descripción
3. **Comentar cada línea** describiendo lo que se está haciendo.
4. **Generar y guardar diagrama de flujo.**

OPERADORES ARITMÉTICOS: MOD

- El operador de modulo MOD, calcula el **residuo de una division entera**.

$$\begin{array}{r} 10 \\ 2 \overline{) 21} \\ \underline{20} \\ 1 \end{array}$$

Residuo

- Ejemplos:

- $21 \text{ MOD } 2 = 1$, porque 2 cabe 10 veces en 21 y sobra 1.
- $10 \text{ MOD } 3 = 1$, porque 3 cabe 3 veces en 10 y sobra 1.
- $7 \text{ MOD } 4 = 3$, porque 4 cabe 1 vez en 7 y sobra 3.
- $10 \text{ MOD } 2 = 0$, porque 10 entre 2 es exacto y no sobra nada.
- $5 \text{ MOD } 10 = 5$, porque el 10 no cabe ninguna vez en el 5 y el total del dividendo pasa a ser el residuo.

OPERADORES ARITMÉTICOS: MOD

Ejemplo 1

Proceso OperacionesEntreEnteros

Definir x1,x2 **Como** Entero

Escribir "Proporciona un valor para x1"

Leer x1

Escribir "Proporciona un valor para x2"

Leer x2

Escribir "El cociente de la división entera entre ",x1," y ",x2," es ",trunc(x1/x2)

Escribir "El residuo de la división entera entre ",x1," y ",x2," es ",x1 mod x2

FinProceso

OPERADORES RELACIONALES

Sirven para formar expresiones lógicas cuyo valor es **FALSO** o **VERDADERO**.

A estos valores también se les conoce como **valores de verdad** o **de certeza**.

Símbolo matemático	Nombre	Símbolo en pseudocódigo
$<$	Menor que	$<$
$\leq$	Menor o igual que	$\leq$
$>$	Mayor que	$>$
$\geq$	Mayor o igual que	$\geq$
$=$	Igualdad lógico	$=, ==$
$\neq$	Desigualdad lógica	$<>, !=$

OPERADORES RELACIONALES

Ejemplo 1:

Considere la siguiente expresión: $a < b$

Cuyo valor puede ser FALSO o VERDADERO, dependiendo del valor de a y b

a	b	$a < b$
4	10	VERDADERO
5	1	FALSO
5	5	FALSO

OPERADORES RELACIONALES

Ejemplo 2:

Considere la siguiente expresión: $a \geq 'f'$

Donde se compara el contenido de una variable con respecto a un caracter.

La variable a puede ser de tipo **entero** o **carácter**, en ambos se utiliza para la comparación con el orden establecido en la tabla de códigos ASCII.

El código ASCII de 'f' es 102.

a	a >= 'f'
'c'	FALSO
'r'	VERDADERO
20	FALSO

OPERADORES LÓGICOS

A las expresiones anteriores se les conoce como **expresión lógica simple**. Usando expresiones lógicas simples, podemos obtener **expresiones lógicas compuestas**.

Una expresión lógica compuesta se forma utilizando operadores relacionales y los operadores lógicos **AND, OR y NOT**.

Los operadores AND y OR requieren dos operandos lógicos, mientras que el operador NOT requiere un solo operando; dichos operandos generalmente surgen como resultado de evaluar expresiones lógicas simples, y dan como resultado otro valor de verdad.

Nombre	Símbolo pseudocódigo
AND	Y, &, &&
OR	O, ,
NOT	NO, !

OPERADORES LÓGICOS: AND

Tabla de verdad del operador lógico AND.

a	b	a Y b
V	V	V
V	F	F
F	V	F
F	F	F

Por ejemplo, considere la expresión compuesta:

$(a > b) \text{ Y } (c \leq d)$

Si $a=10$, $b=3$, $c=1$, y $d=1$

$\begin{array}{cc} \text{v} & \text{v} \\ \hline & \text{v} \end{array}$

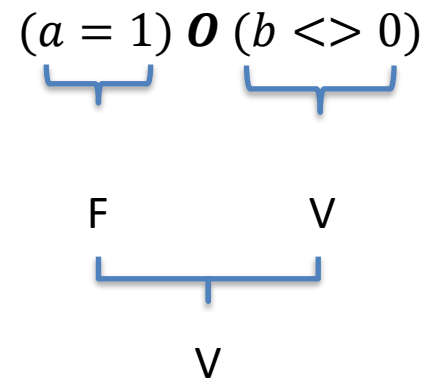
OPERADORES LÓGICOS: OR

Tabla de verdad del operador lógico OR.

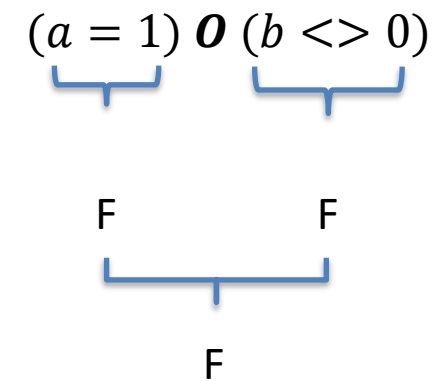
a	b	a O b
V	V	V
V	F	V
F	V	V
F	F	F

Por ejemplo, considere la expresión compuesta:

Si $a=7$, $b=15$,



¿Y si $a=6$, $b=0$?



OPERADORES LÓGICOS: NOT

Tabla de verdad del operador lógico NO.

x	NO x
V	F
F	V

Por ejemplo, la expresión:

NO ($a < 6$)

Si $a=10$  V

Si $a=1$  F

OPERADORES LÓGICOS: LEYES de DE MORGAN

Leyes de DeMorgan para la lógica.

$\text{NO}(x1 \text{ Y } x2) \equiv \text{NO } x1 \text{ O NO } x2$
$\text{NO}(x1 \text{ O } x2) \equiv \text{NO}x1 \text{ Y NO } x2$

Hacen posible escribir una expresión lógica compuesta de **dos formas diferentes** y, sin embargo, ser **equivalentes**.

Por ejemplo:

$$\begin{aligned} & \text{NO}(x+y>0 \text{ Y } x\geq 0) \\ & \equiv \text{NO}(x+y>0) \text{ O NO}(x\geq 0) \\ & \equiv (x+y\leq 0) \text{ Y } (x<0) \end{aligned}$$

PRECEDENCIA DE OPERADORES (actualización)

NO(x+y>0 Y x>=0)

En la expresión anterior, usamos paréntesis en cada expresión donde se aplica una operación **NOT** para indicar que ésta, será evaluada después de la expresión que aparece dentro de ellos.

Pero no utilizamos paréntesis para indicar que la suma debe realizarse antes de la comparación en la expresión x+y>0. Esto se debe a que el operador + tiene mayor precedencia que >.

Precedencia de los operadores.

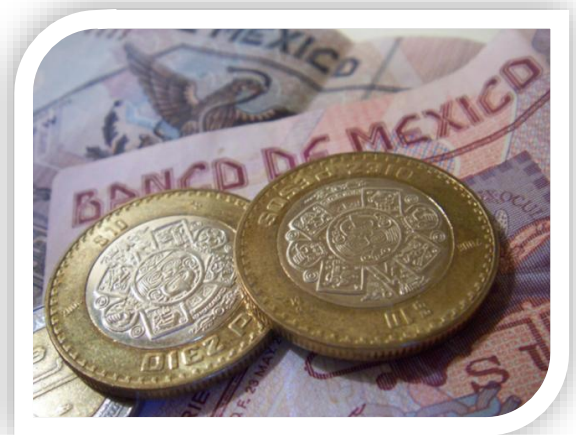
Operadores			
()			
NOT	+	-	(monarios)
* / MOD			
+ - (binarios)			
< <= > >=			
<> =			
Y, &, &&			
O, ,			

Práctica 5a: Separar dinero

PROBLEMA: Separar una cantidad en billetes de \$100, monedas de \$10 y de \$1 peso.

INSTRUCCIONES:

1. Leer la cantidad que se desea descomponer:
`cantidad`
2. Para separar en billetes de 100, hay que dividir la cantidad entre 100 con división entera:
`billetes100 := trunc (cantidad/100)`
3. Una vez obtenido el número de billetes de 100, hay que calcular el resto del dinero:
`resto = cantidad mod 100`
4. Con el resto se calcula el número de monedas de 10 que se requieren usando división entera:
`monedas10 := trunc (resto/10)`
5. Finalmente, se obtiene el número de monedas de 1 peso requeridas:
`monedas1 = resto mod 10`



Práctica 5b: Separar dinero, más denominaciones

PROBLEMA: Resuelve el ejercicio anterior, pero ahora incluye también las siguientes denominaciones:

- Billetes de 50
- Billetes de 20
- Monedas de 5

- No olvidar incluir cabecera:
 - Nombre del archivo:
 - Autor:
 - Descripción
- Incluir comentarios por línea
- Generar diagrama de flujo



Práctica 6: Operadores relacionales y lógicos

INSTRUCCIONES:

1. Pedir al usuario valores para: a y b (una sola vez)
2. Crear las siguientes expresiones lógicas compuestas:
 - a) $(a > b) \text{ Y } (b > a)$
 - b) $(a > b) \text{ O } (b > a)$
 - c) $\text{NO}(a > b)$
 - d) $(a \neq b) \text{ Y } (b > 5)$
 - e) $(a \neq b) \text{ O } (b > 5)$
 - f) $\text{NO}(a \neq b)$
3. Imprimir en pantalla cada expresión, sustituyendo a y b por los valores dados por el usuario y mostrando el valor de verdad resultante para cada expresión.

Ejemplo a):

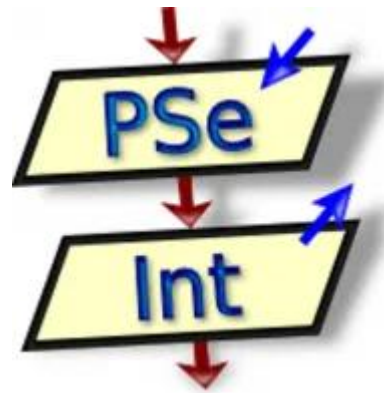
```
a = 5
b = 10
(5 > 10) Y (10 > 5) es Falso
```

Práctica 7: Operadores relacionales y lógicos

INSTRUCCIONES:

1. Pedir al usuario valores para 4 variables enteras a , b , c y d .
2. Crear las siguientes expresiones lógicas:
 - a) Los valores de a y b son ambos menores que 17.
 - b) Los valores de a , b y c son idénticos y distintos de d .
 - c) El valor de b es mayor que el valor de a y menor que c .
 - d) Entre los valores de a , b , y c hay al menos dos idénticos.
3. Imprimir en pantalla cada expresión, sustituyendo a , b , c y d por los valores dados por el usuario y mostrando el valor de verdad resultante para cada expresión. (Como en el ejercicio anterior).

Estructuras de Control en PSeInt



ESTRUCTURAS DE CONTROL

Hasta el momento, hemos visto ejercicios donde se inicia con la ejecución de la primera instrucción, seguido de la ejecución de la segunda instrucción, etc. Y además, cada una de las instrucciones se ejecuta una sola vez.

A estos algoritmos se les conoce como **algoritmos secuenciales**.

Esta forma de programar es útil cuando se trata de problemas relativamente sencillos, pero generalmente, es necesario utilizar otros mecanismos que nos permitan decidir entre la ejecución de una instrucción (o un conjunto de instrucciones) y otra, así como para permitir que se ejecuten en más de una ocasión.

ESTRUCTURAS DE CONTROL

Esto es precisamente lo que harán las estructuras de control:

Elegir qué instrucciones se van a ejecutar cada vez.

Existen tres tipos de estructuras de control:

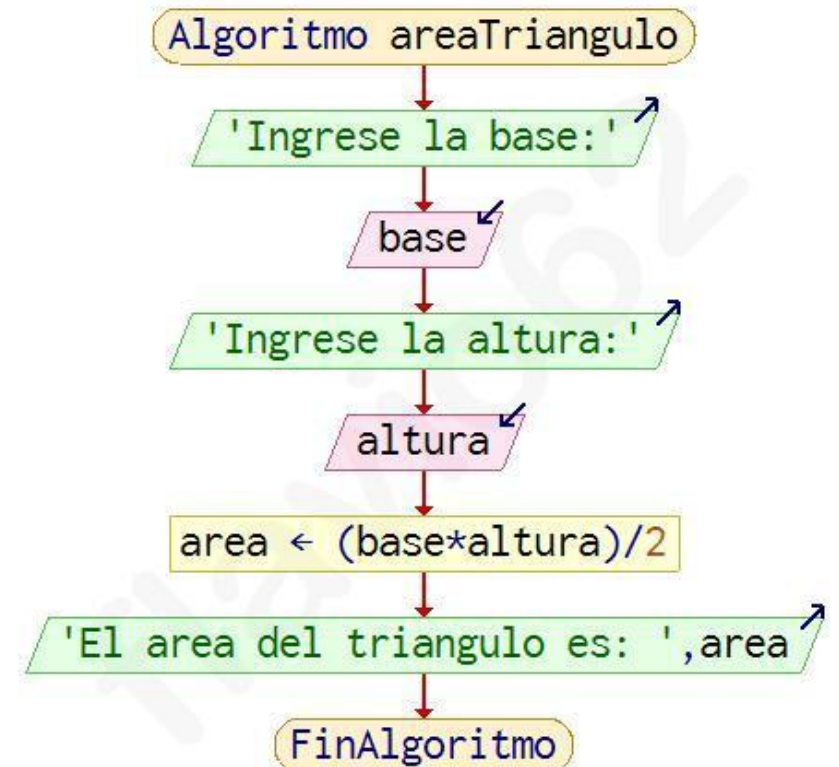
Secuenciación
(secuenciales)

Selección
(condicionales)

Repetición
(iterativas)

SECUENCIACIÓN (secuenciales)

- Las instrucciones se ejecutan **una tras otra**, en **orden lineal y continuo**.
- **No existen saltos ni decisiones**, por lo tanto, el flujo va del primer paso al último.



SELECCIÓN (condicionales)

Permite seleccionar el **curso de ejecución** de algunas instrucciones del programa en base al cumplimiento de alguna **condición**.

Existen tres tipos de selección:

Simple

Doble

Múltiple

SELECCIÓN SIMPLE

Es una estructura de control que permite **decidir si se ejecuta o no**, un conjunto de instrucciones.

Dicha estructura se vale de una **expresión lógica (simple o compuesta)** llamada ***condición*** para decidir el flujo de acciones que se ejecutarán.

El formato es el siguiente:

Si *condición* **Entonces**
 ↳ Bloque de instrucciones a realizar cuando *condición* es verdadera
FinSi

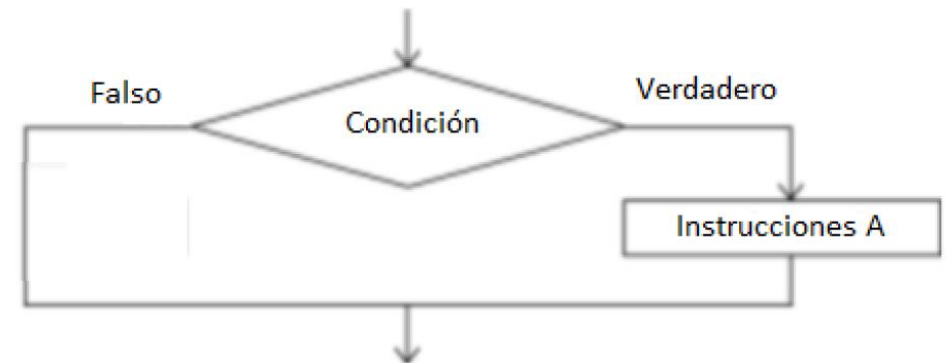
identación

SELECCIÓN SIMPLE

El flujo de acciones es el siguiente:

- Si *condición* es VERDADERO, entonces se ejecuta el bloque de instrucciones dentro de la estructura.
- En caso contrario se ignora.

El fin del bloque de instrucciones a ejecutar cuando *condición* es VERDADERO se especifica con la sentencia **FinSi**.



SELECCIÓN SIMPLE

Ejercicio: Felicitar si es mayor de edad

INSTRUCCIONES:

- Leer el nombre y la edad del usuario.
- Saludarle por su nombre.
- Si el usuario tiene 18 años, mandarle un mensaje de felicitación por se mayor de edad.

1. Identificar:

- Entrada: *Leer el nombre y la edad del usuario*
- Proceso: *Mostrar el nombre y comparar si edad es igual a 18*
- Salida: *Si edad es igual a 18 se manda un mensaje felicitando al usuario*

2. Dibujar diagrama de flujo



SELECCIÓN SIMPLE: Ejercicio

Diagrama de flujo



Práctica 8:

Felicitar si es mayor de edad



INSTRUCCIONES:

- Leer el nombre y la edad del usuario.
- Saludarlo por su nombre.
- Si el usuario tiene 18 años, mandarle un mensaje de felicitación.

En el cuaderno ya hicimos:



- ✓ Entrada
 - ✓ Proceso
 - ✓ Salida
-
- ✓ Diagrama de flujo

En PSeInt:

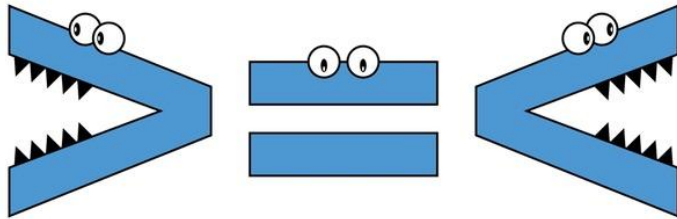


1. Escribir pseudocódigo.
 - a) Cabecera (Nombre archivo, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo
3. Guardar sus archivos.

Ejercicio: Menor de dos números (usando solo selección simple)

PROBLEMA:

Suponga que se tienen dos números enteros y se desea compararlos para imprimir en pantalla cuál es el menor de ellos.



En el cuaderno:



1. Identificar:

- Entrada: *Leer num1 y num2*
- Proceso: *Comparar si:*
 - a) $num1 < num2$
 - b) $num2 < num1$

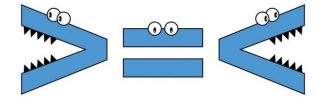
¿Y si son iguales?

 - c) $num1 = num2$
- Salida: *Mostrar mensaje indicando cuál número es menor, o si son iguales.*

2. Dibujar diagrama de flujo

Práctica 9:

Menor de dos números (usando sólo selección simple)



PROBLEMA:

Suponga que se tienen dos números enteros y se desea compararlos para imprimir en pantalla cuál es el menor de ellos.

En el cuaderno ya hicimos:



- ✓ Entrada
- ✓ Proceso
- ✓ Salida

- ✓ Diagrama de flujo

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 10:

Determinar si un número entero es par (usando sólo selección simple)

PROBLEMA:

Se le pide un número entero al usuario y se quiere indicar en pantalla si es par o impar.

En el cuaderno:



1. **Identificar:**
 - a) Entrada
 - b) Proceso
 - c) Salida
2. **Diagrama de flujo**

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**